LING285-L10-9-24

Course: LING285: Human Language and Technology
Title of Lecture: Neural Networks 1.5
Date: 9/24/2026
Professor: Khalil Iskarous

Section 1: The Job of a Neural Network

- Two recognition problems show what a **neural network (NN)** is built to do. In the first, we are handed a huge number of pictures of faces, each made of about **48 million pixels**, and we have to say which of **1000 possible people** each face belongs to. In the second, we are handed **speech waves**, and for each wave we have to say which of **25 sounds** it corresponds to. A neural network is the kind of system that can solve both of these problems.
    - A pixel is one tiny dot of an image, and each pixel is stored as a number that describes its brightness or color. So a single face picture is really a list of about 48 million numbers.
    - A speech wave is also stored as a list of numbers, one for each moment in time, so the speech problem has the same basic form as the face problem. Numbers go in, and a choice among a fixed set of options has to come out.
- The name "neural network" is not explained yet. The network is instead defined by its behavior, meaning by what it takes in and what it gives back. Its job is to take a bunch of numbers, such as the 48 million pixel values of one image or the numbers that make up one sound wave, and return a **probability distribution**, which is written here as a **pdf**.
    - A probability distribution is a list of probabilities, one for each possible answer. Each probability says how likely the network thinks that answer is.
    - For the speech problem with 25 sounds, the distribution has 25 numbers, one per sound. For the face problem with 1000 people, it has 1000 numbers, one per person.
- To keep the example small, suppose there are only **5 vowels** instead of 25 sounds. We agree on a fixed order for the vowels: the vowel in **read**, the vowel in **rad**, the vowel in **rude**, the vowel in **Rud**, and the vowel in **rod**. A possible pdf for one speech wave is then:

$$[0.2,\ 0.2,\ 0.4,\ 0.2,\ 0]$$

- Each position lines up with one vowel in the agreed order. This pdf says there is a 20% chance the sound is the vowel in read, a 20% chance it is the vowel in rad, a 40% chance it is the vowel in rude, a 20% chance it is the vowel in Rud, and a 0% chance it is the vowel in rod.
- The agreed order matters. The numbers carry no meaning unless everyone knows which position belongs to which vowel.
- The probabilities in a pdf **must sum to 1**, which is the same as 100%. That rule holds for the example above:

$$0.2 + 0.2 + 0.4 + 0.2 + 0 = 1.0$$

- The sum has to be 1 because the sound must be one of the five vowels. All of the possible answers together have to account for all of the chance.
- In this example, the network's best guess is the vowel in rude, since 0.4 is the largest number. It is still not certain, since 60% of the probability is spread over other vowels.

Section 2: Inputs, Weights, and How the Network Learns

- The **input** of a neural network is a bunch of numbers, which can also be called a **signal** or a **vector**. A vector is just an ordered list of numbers, such as $[4, 1, -3, 2, 1, 0.4, 1, 2, -3, \ldots]$. Real inputs usually contain thousands of numbers. In the speech case, these could be the **pressure numbers of a speech wave**, meaning the air pressure measured at each moment as the sound passes.

- The **output** of the network is a **decision pdf**. Given the input signal, the network returns the probability of each possible object, which here is the probability of each of the vowels in read, rad, rude, Rud, and rod.
- The network can make this decision because it has **knowledge**, and that knowledge is stored in a large set of numbers called **weights**, written w. The input vector is combined with these weights through many calculations, and at the end a decision about which object was detected comes out.
    - The scale is enormous. Large networks have **several trillion weights**, and making one decision involves **billions of calculations**.
    - The weights are what separate a network that decides well from one that decides badly. Two networks with the same structure but different weights will give different answers to the same input.
- The weights are not set by hand, so the network has to find them on its own. The strategy is **"Fake it till you make it" (FITYMI)**, and the method that carries it out is **stochastic gradient descent (SGD)** applied to neural networks. The procedure runs as a loop:
    1. We start the network with **random weights**. At this point the weights contain no real knowledge.
    2. We give the network one signal. Suppose it is a recording of the vowel in **Rud**.
    3. The network runs **forward propagation**, which means it pushes the input through all of its calculations using its current weights and produces a pdf. With random weights, that pdf does not reflect reality. For example, it might return [0.4, 0.1, 0.2, 0.2, 0.1].
    4. We tell the network that its answer is wrong. The correct pdf is [0, 0, 0, 1, 0], meaning 100% on the vowel in Rud and 0% on everything else. The gap between what the network said and what it should have said is its **error**, also called its **loss**.
    5. The network adjusts its weights so that the loss becomes smaller.
    6. We give it another signal and repeat the whole loop, **billions of times**.
- The wrong pdf in step 3 shows why the loss is large. The network put 0.4 on the vowel in read, its highest value, and only 0.2 on the vowel in Rud, which is the true answer. Its numbers still sum to 1:

$$0.4 + 0.1 + 0.2 + 0.2 + 0.1 = 1.0$$

- So the network gives a valid probability distribution, but the probability is in the wrong places. Learning means moving probability toward the correct answer, so that after many rounds the network puts nearly all of it on the vowel that was actually spoken.
- The correct pdf, [0, 0, 0, 1, 0], is a distribution with all of its probability on a single answer. It serves as the target the network is pushed toward.

Section 3: Two Huge Questions — Learning and Inference

- Two big questions describe how a neural network works.
    - **Learning** asks how, specifically, a network learns its weights.
    - **Inference** asks how, specifically, a network combines its weights with its inputs to produce an output.
- Inference is answered first, since inference is part of learning. Every round of learning begins with forward propagation, and forward propagation is inference. The network cannot measure its loss until it has combined the input with its current weights and produced an answer. So the exact calculation used in inference has to be understood before learning can be.

Section 4: Neurons, Activity, and Synaptic Weights

- **Santiago Ramón y Cajal** established that **the brain is made of neurons**, meaning separate individual cells rather than one continuous web. The figure here is **Cajal's drawing of a retinal circuit**, from *Recuerdos de mi vida*, volume II (1917), Figure 40. The drawing is cropped to one half and rotated so that the main direction of signal flow runs from left to right.

    - In the drawing, circles mark the **somata** of the neurons. A soma is the cell body, the central part of the neuron that holds the nucleus.
    - Each neuron is given a number called its **activity** or **activation**, and these numbers can be positive or negative. Negative values here are quantities used in artificial neurons. They do not mean that a real neuron fires at a negative rate, which is impossible.
    - Arrows mark selected **synaptic contacts**, the points where one neuron connects to another. Signals travel rightward through these contacts, from the neurons on the left to a receiving neuron on the right.
    - Each synaptic contact is then labeled with a **weight** w, and each w is replaced by a signed number. A **synaptic weight** says how strongly, and in which direction, one neuron's activity affects the next neuron. A positive weight pushes the receiving neuron's activity up, and a negative weight pushes it down.
- The circuit is then reduced to its simplest form: **three input neurons** and **one receiving neuron**. The three connections carry weights of **−2**, **3**, and **1**. The three input neurons have activations of **−1**, **1**, and **3**.
    - The first input, with activation −1, connects through weight −2.
    - The second input, with activation 1, connects through weight 3.
    - The third input, with activation 3, connects through weight 1.
- **One neuron computes a weighted sum.** The receiving neuron multiplies each input activation by the weight of its connection and then adds up all of the products:

$$(-1 \times -2) + (1 \times 3) + (3 \times 1) = 8$$

- The first product is $(-1)(-2) = 2$. Multiplying a negative activation by a negative weight gives a positive result, so this input actually raises the receiving neuron's activity.
- The second product is $(1)(3) = 3$, and the third is $(3)(1) = 3$.
- Adding them gives $2 + 3 + 3 = 8$. That number, **8**, becomes the activation of the receiving neuron. No extra constant, called a **bias**, is added in this example.
- This weighted sum has several names, and all of them mean the same operation: the **inner product**, the **scalar product**, and the **dot product**. Each takes two lists of the same length, here the list of activations $[-1, 1, 3]$ and the list of weights $[-2, 3, 1]$, multiplies them position by position, and adds the results into one number.
    - It is called the scalar product since its result is a single number, and a single number is called a scalar.
    - The same idea also appears as $\int f \cdot g$. When the two lists are replaced by two continuous functions f and g, multiplying position by position becomes multiplying the functions point by point, and adding up the products becomes taking an integral. So the integral of a product of two functions is the continuous version of the dot product.
- This operation is the answer to the inference question at the level of one neuron. A neuron in an artificial network combines its inputs with its weights by taking the dot product of the incoming activations and the incoming weights.

Section 5: From Brain Chemistry and Electronics to Arithmetic

- The brain works through chemistry and electrical activity, while neural networks work through arithmetic. Three publications mark the turning of brain activity into arithmetic.
    - **1943: Warren S. McCulloch and Walter Pitts**, "A Logical Calculus of the Ideas Immanent in Nervous Activity," written at the University of Illinois College of Medicine, the Department of Psychiatry at the Illinois Neuropsychiatric Institute, and the University of Chicago. This paper treated nervous activity as something that could be described with logic and calculation.
    - **1962: Frank Rosenblatt**, *Principles of Neurodynamics: Perceptrons and the Theory of Brain Mechanisms*, published by Spartan Books. This book developed the **perceptron**, an artificial neuron that computes a weighted sum of its inputs and uses it to make a decision.
    - **1986: David E. Rumelhart, James L. McClelland, and the PDP Research Group**, *Parallel Distributed Processing: Explorations in the Microstructure of*

Cognition, Volume 1: Foundations. This work described cognition as the activity of many simple neuron-like units working at the same time, with knowledge spread across the weights between them. It also presented backpropagation, the learning method used to train networks with many layers.

- Over these four decades, the electrical and chemical activity of neurons came to be described as numbers, and the passing of signals across synapses came to be described as multiplying by weights and adding. That is the same weighted-sum calculation performed by a single artificial neuron.

Section 6: Two Beliefs About AI and the Brain

- There are two opposing beliefs about how modern AI relates to the human mind. These are views, not exam material.
  - Most engineers, neuroscientists, and people in general hold that modern AI has nothing to do with how humans think, perceive, act, or decide. On this view, the brain is full of chemistry, electrical activity, and abstract processes, and none of that is part of **large language models (LLMs)**, which are made only of numbers and arithmetic.
  - A smaller group, including Khalil Iskarous, holds that modern AI is the result of about 100 years of gradual, evolutionary tinkering with how arithmetic can **emulate the chemical and electrical activity of the brain**. On this view, those attempts appear to have succeeded, so the arithmetic in modern AI is a working imitation of brain activity rather than something unrelated to it.

[CONTINUE]

Section 7: A Multilayer Perceptron

- A **multilayer perceptron (MLP)** is a neural network built from **layers** of artificial neurons, where every neuron in one layer connects to every neuron in the next layer. The example network has five layers in a row: an **Input** layer, **Hidden 1**, **Hidden 2**, **Hidden 3**, and an **Output** layer. The layers in the middle are called **hidden layers**, since they are neither the input we give the network nor the answer it gives back.
  - The layer sizes are **5, 3, 4, 3, and 7**. The input layer has 5 neurons, Hidden 1 has 3, Hidden 2 has 4, Hidden 3 has 3, and the output layer has 7.
  - Every connection between neighboring layers carries its own weight w. The number of connections comes from multiplying the sizes of each pair of neighboring layers and adding:

$$(5 \times 3) + (3 \times 4) + (4 \times 3) + (3 \times 7) = 15 + 12 + 12 + 21 = 60$$

- So this small network has **60 weights**. Real networks follow the same design but with far more neurons per layer, which is how the count of weights reaches the trillions.
- Every neuron in a hidden layer does exactly what the single neuron in Cajal's circuit did. It receives activations from all neurons in the layer before it, multiplies each by the weight of its connection, and adds the products. This is the dot product again, now repeated for every neuron in every layer.
- The five input neurons carry the activations **2, 1, 3, 4, and 2**. The middle neuron of Hidden 1 receives five connections, one from each input neuron, with weights **2, −1, 1, 3, and −2**. Its activation is the dot product of the inputs with those weights:

$$(2 \times 2) + (1 \times -1) + (3 \times 1) + (4 \times 3) + (2 \times -2)$$

$$= 4 + (-1) + 3 + 12 + (-4) = 14$$

- The largest contribution comes from the fourth input. Its activation of 4 is multiplied by a weight of 3, giving 12.
- The two negative weights pull the total down. The second input contributes −1 and the fifth input contributes −4.
- The result, **14**, is the activation of the middle neuron in Hidden 1.
- The other two neurons in Hidden 1 compute their own dot products with their own sets of five weights. The top neuron uses the weights 0.5, 1, 1, −0.5, and 0.5, and the bottom

neuron uses −1, 0.5, 0.5, −1, and 0.5. The three activations of Hidden 1 come out as **4, 14, and −3**.

$$(2 \times 0.5) + (1 \times 1) + (3 \times 1) + (4 \times -0.5) + (2 \times 0.5) = 1 + 1 + 3 - 2 + 1 = 4$$

$$(2 \times -1) + (1 \times 0.5) + (3 \times 0.5) + (4 \times -1) + (2 \times 0.5) = -2 + 0.5 + 1.5 - 4 + 1 = -3$$

- The same calculation then moves forward one layer at a time. Hidden 1's activations, 4, 14, and −3, become the inputs to Hidden 2. For example, the top neuron of Hidden 2 has weights 1, 0.25, and 0.5 on its three incoming connections:

$$(4 \times 1) + (14 \times 0.25) + (-3 \times 0.5) = 4 + 3.5 - 1.5 = 6$$

- The four neurons of Hidden 2 end up with the activations **6, −2, 5, and 1**.
- Those four numbers then feed Hidden 3, whose three neurons end up with the activations **3, −1, and 2**.
- Passing activations from layer to layer this way, from the input all the way to the output, is **forward propagation**. It is the full answer to the inference question: the network combines its weights with its inputs by computing dot products, layer after layer.
- The hidden layers here use **linear hidden activations**, meaning each neuron's activation is simply its weighted sum with nothing further done to it. No bias is added either. A usual MLP adds one more step at each hidden neuron, a **nonlinear activation function**, which reshapes the weighted sum before passing it on. That step is left out here so the arithmetic stays pure dot products.
- The output layer has 7 neurons, and its final values are **0.05, 0.10, 0.20, 0.30, 0.15, 0.12, and 0.08**. These come from a **softmax** step. Each output neuron first computes a weighted sum of Hidden 3's activations, which gives a raw score. Softmax then converts the seven raw scores into seven positive numbers that sum to 1, with larger scores turning into larger probabilities.

$$0.05 + 0.10 + 0.20 + 0.30 + 0.15 + 0.12 + 0.08 = 1.00$$

- Softmax is what turns the network's arithmetic into the **decision pdf** that a neural network is supposed to return.
- In this example, the fourth output gets the highest probability, 0.30, so it is the network's best guess among the seven possible answers.

Section 8: Pop Quiz — The Activation on Hidden 1, Neuron 2

- The same network is given a new input. The five input neurons now carry the activations **−2, 1, 3, −4, and 2**, and the five weights leading into the second neuron of Hidden 1 are **3, −2, 1, 2, and 5**. The question is the activation of Hidden 1, Neuron 2.
- The activation is the dot product of the inputs and the weights:

$$(-2 \times 3) + (1 \times -2) + (3 \times 1) + (-4 \times 2) + (2 \times 5)$$

$$= -6 + (-2) + 3 + (-8) + 10 = -3$$

- The first input, −2, times the weight 3 gives −6.
- The second input, 1, times the weight −2 gives −2.
- The third input, 3, times the weight 1 gives 3.
- The fourth input, −4, times the weight 2 gives −8.
- The fifth input, 2, times the weight 5 gives 10.
- Adding these gives $-6 - 2 + 3 - 8 + 10 = -3$. The activation of Hidden 1, Neuron 2 is **−3**.
- Since the hidden activations are linear, the answer is just the weighted sum, with no bias added and no further function applied. The output neurons stay empty until every layer before them has been computed, since each layer depends on the one before it.

Section 9: Learning by FITYMI — Backpropagation

- Learning follows the "Fake it till you make it" plan. We start with **random weights** in the multilayer perceptron, and we randomly pick one **input-output pair (I/O pair)** to learn

from. An I/O pair is one input together with the correct answer for that input. Here the input is $x = [-2, 1, 3, -4, 2]$, and the correct output is:

$$[0,\ 0,\ 0,\ 0,\ 0,\ 1,\ 0]$$

- This target pdf puts all of its probability on the sixth of the seven possible answers. So the sixth answer is correct, and the other six are wrong.
- With random weights, forward propagation produces a pdf that is **wrong**. Suppose the network returns 0.05, 0.10, 0.20, 0.30, 0.15, 0.12, and 0.08. It gives only 0.12 to the sixth answer, which should have received 1, and it gives its highest probability, 0.30, to the fourth answer, which should have received 0.
    - The difference between this pdf and the true pdf is the network's loss for this I/O pair.
- The network then **changes its weights to make its output pdf closer to the true one**. The method is **SGD**, which in neural networks is now called **backpropagation**. The error measured at the output is passed backward through the layers, and each of the 60 weights is adjusted in the direction that makes the loss smaller.
    - After one adjustment, the network with this same input would give a little more probability to the sixth answer and a little less to the others.
    - One adjustment does not make the network correct. It only makes it slightly less wrong.
- The process is **repeated over and over again with billions of I/O pairs**. Each pair gives one small correction to the weights, and over billions of corrections, the weights come to hold real knowledge about which inputs go with which outputs. This repeated training on billions of pairs is the work carried out at **data centers**.

Section 10: Python SGD for an Average

- SGD can be applied to a much simpler problem: estimating the **average** of a list of numbers. Here the weight w is a single number, our estimate of the average of this list:

$$[73,\ 90,\ 98,\ 46,\ 400,\ 81,\ 82,\ 78,\ 62,\ 91,\ 61,\ 84,\ 75,\ 84,\ 81,\ 75]$$

- The network in this case has just one weight, and learning means finding the value of w that best represents the whole list.
- One of the numbers, **400**, is a mistake. Every other number lies between 46 and 98, so 400 does not belong with the rest of the data. Even so, it strongly affects the average. The full list has 16 numbers, and their sum is 1561:

$$\frac{1561}{16} \approx 97.56$$

- So the actual average of the list, including the mistaken 400, is **about 100**.
- Without the mistake, the true average is **about 80**. Removing 400 leaves 15 numbers with a sum of $1561 - 400 = 1161$:

$$\frac{1161}{15} = 77.4$$

- A single bad data point raises the average by about 20. An estimate of w learned from this data will be pulled toward the mistake, just as a neural network trained on data that contains errors will learn some of those errors.
- SGD sees data points one at a time in a **random order**. For simplicity, the list is assumed to already be in the random order in which the data will arrive. So w is first compared against 73, then against 90, then against 98, and so on through the list. After each number, w is adjusted to reduce its error on that number, following the same loop the neural network uses on its I/O pairs.

Course: LING285: Human Language and Technology
Title of Lecture: Neural Networks 1.5
Date: 9/24/2026
Professor: Khalil Iskarous

Section 1: The Job of a Neural Network

- A **neural network (NN)** can solve two kinds of recognition problems. In one, we are given many pictures of faces, each made of about **48 million pixels**, and we have to identify which of **1000 possible people** each face belongs to. In the other, we are given **speech waves**, and we have to identify which of **25 sounds** each wave corresponds to.
    - Each pixel is stored as a number, so one face picture is a list of about 48 million numbers. A speech wave is also stored as a list of numbers.
- A neural network is defined here by what it does rather than by why it has its name. Its job is to take a bunch of numbers, such as the 48 million pixels of one image or the numbers of one sound wave, and return a **probability distribution**, written **pdf**. For the speech problem, the pdf gives one probability for each of the 25 sounds.
- Suppose there are only **5 vowels**, and we agree on the order **read, rad, rude, Rud, rod**. One possible pdf is:

$$[0.2,\ 0.2,\ 0.4,\ 0.2,\ 0]$$

- This pdf says there is a 20% chance the sound is the vowel in read, 20% the vowel in rad, 40% the vowel in rude, 20% the vowel in Rud, and 0% the vowel in rod.
- The numbers only have meaning once the order of the vowels is agreed on, since each position stands for one vowel.
- The probabilities in a pdf **must sum to 1**, which is 100%:

$$0.2 + 0.2 + 0.4 + 0.2 + 0 = 1.0$$

- The largest value, 0.4, falls on the vowel in rude, so rude is the most likely answer under this distribution.

Section 2: Inputs, Weights, and Learning by FITYMI

- The **input** of a neural network is a bunch of numbers, also called a **signal** or a **vector**, such as $[4, 1, -3, 2, 1, 0.4, 1, 2, -3, \ldots]$. A vector is an ordered list of numbers, and real inputs usually contain thousands of them. In the speech case, these can be the **pressure numbers of a speech wave**.
- The **output** is a **decision pdf**. Given the signal, the network returns the probability of each object, here the probability of each of the vowels in read, rad, rude, Rud, and rod.
- The network makes decisions using **knowledge stored in weights**, written w. The input vector is combined with these weights through many calculations, and a decision about the detected object comes out at the end. Large networks have **several trillion weights** and carry out **billions of calculations**.
- The network finds its weights through **"Fake it till you make it" (FITYMI)**, carried out by **stochastic gradient descent (SGD)** for neural networks. The procedure is a repeated loop:
    1. The network starts with **random weights**.
    2. It is given one signal, for example the vowel in **Rud**.
    3. It runs **forward propagation**, using its current weights to produce a pdf. With random weights, the pdf does not reflect reality, for example [0.4, 0.1, 0.2, 0.2, 0.1].
    4. It is told the correct pdf, [0, 0, 0, 1, 0], which puts 100% on the vowel in Rud. The gap between its answer and the correct answer is its **error**, also called its **loss**.
    5. It improves its weights to reduce the loss.
    6. It is given another signal, and the loop repeats **billions of times**.
- The wrong pdf in step 3 is still a valid distribution, since it sums to 1:

$$0.4 + 0.1 + 0.2 + 0.2 + 0.1 = 1.0$$

- Its mistake is where the probability sits. It places its highest value, 0.4, on the vowel in read, and only 0.2 on the vowel in Rud, which should have received 1.

Section 3: Two Huge Questions — Learning and Inference

- **Learning** asks how, specifically, a network learns its weights. **Inference** asks how, specifically, a network combines its weights with its inputs.

- Inference is answered first, since inference is part of learning. Each round of learning begins with forward propagation, which is inference, and the loss can only be measured after the network has combined the input with its weights to produce an answer.

Section 4: Neurons, Activity, and Synaptic Weights

- **Santiago Ramón y Cajal** showed that the **brain is made of neurons**. His drawing of a **retinal circuit**, from *Recuerdos de mi vida*, volume II (1917), Figure 40, is cropped and rotated so that signals flow from left to right.
  - Circles mark the **somata**, the cell bodies of the neurons.
  - Each neuron carries an **activation**, a number that can be positive or negative. Negative activations are quantities used in artificial neurons, not negative firing rates of real neurons.
  - Arrows mark selected **synaptic contacts**, where one neuron connects to the next, and signals travel rightward through them.
  - Each contact becomes a **synaptic weight** w, and each weight becomes a signed number. A positive weight increases the receiving neuron's activation, and a negative weight decreases it.
- The circuit reduces to **three input neurons** and **one receiving neuron**. The input activations are **−1, 1, and 3**, and the three connections carry the weights **−2, 3, and 1**, in the same order.
- **One neuron computes a weighted sum.** It multiplies each input activation by the weight of its connection and adds the products:

$$(-1 \times -2) + (1 \times 3) + (3 \times 1) = 8$$

- The products are 2, 3, and 3. The first is positive since a negative activation times a negative weight gives a positive number.
- Their sum, **8**, becomes the activation of the receiving neuron. No bias, an extra added constant, is used here.
- This weighted sum is called the **inner product**, the **scalar product**, or the **dot product**. It takes two lists of equal length, here the activations $[-1, 1, 3]$ and the weights $[-2, 3, 1]$, multiplies them position by position, and adds the results into a single number.
  - The same operation is written $\int f \cdot g$ when the two lists are replaced by two continuous functions f and g. The products become the point-by-point product of the functions, and the sum becomes an integral.
- The dot product answers the inference question for one neuron. A neuron combines its inputs with its weights by taking the dot product of the incoming activations and the incoming weights.

Section 5: From Brain Chemistry and Electronics to Arithmetic

- Three publications mark the move from describing the brain as chemistry and electronics to describing it as **arithmetic**.
  - **1943: Warren S. McCulloch and Walter Pitts**, "A Logical Calculus of the Ideas Immanent in Nervous Activity," from the University of Illinois College of Medicine, the Department of Psychiatry at the Illinois Neuropsychiatric Institute, and the University of Chicago.
  - **1962: Frank Rosenblatt**, *Principles of Neurodynamics: Perceptrons and the Theory of Brain Mechanisms*, published by Spartan Books.
  - **1986: David E. Rumelhart, James L. McClelland, and the PDP Research Group**, *Parallel Distributed Processing: Explorations in the Microstructure of Cognition*, Volume 1: Foundations.

Section 6: Two Beliefs About AI and the Brain

- Two opposing beliefs describe how modern AI relates to the human mind, and neither is exam material.
  - Most engineers, neuroscientists, and people in general believe modern AI has nothing to do with how humans think, perceive, act, or decide. On this view, the

brain is full of chemistry, electrical activity, and abstract processes, none of which is part of **LLMs**, which are made only of numbers and arithmetic.
    - A few people, including Khalil Iskarous, believe modern AI is the product of about **100 years of evolutionary tinkering** with how arithmetic can **emulate the chemical and electrical activity of the brain**, and that these attempts appear to have succeeded.

[CONTINUE]

Section 7: A Multilayer Perceptron

- A **multilayer perceptron (MLP)** is a network of artificial neurons arranged in layers, where every neuron in one layer connects to every neuron in the next. This example has five layers: **Input**, **Hidden 1**, **Hidden 2**, **Hidden 3**, and **Output**, with **5, 3, 4, 3, and 7** neurons. Every connection carries its own weight w, which gives 60 weights in total:

$$(5 \times 3) + (3 \times 4) + (4 \times 3) + (3 \times 7) = 15 + 12 + 12 + 21 = 60$$

- Each hidden neuron computes a **dot product**, the same weighted sum as a single neuron in Cajal's circuit. It multiplies the activation of every neuron in the previous layer by the weight of its connection and adds the products.
- The input activations are **2, 1, 3, 4, and 2**. The middle neuron of Hidden 1 receives the five inputs through the weights **2, −1, 1, 3, and −2**:

$$(2 \times 2) + (1 \times -1) + (3 \times 1) + (4 \times 3) + (2 \times -2)$$

$$= 4 + (-1) + 3 + 12 + (-4) = 14$$

- The fourth input gives the largest contribution, 12, and the two negative weights subtract 1 and 4 from the total.
- Repeating this calculation layer by layer gives the activations of every layer. Hidden 1 has the activations **4, 14, and −3**. These become the inputs to Hidden 2, which has the activations **6, −2, 5, and 1**. These in turn feed Hidden 3, which has the activations **3, −1, and 2**. Passing activations forward from input to output in this way is **forward propagation**, and it is the full answer to the inference question.
- The hidden layers use **linear hidden activations**, meaning each neuron's activation is just its weighted sum, and no bias is added. A usual MLP adds a **nonlinear activation function** at each hidden neuron, a step left out here.
- The output layer uses **softmax**, which turns the output neurons' weighted sums into probabilities that **sum to 1.00**. The seven output probabilities are **0.05, 0.10, 0.20, 0.30, 0.15, 0.12, and 0.08**:

$$0.05 + 0.10 + 0.20 + 0.30 + 0.15 + 0.12 + 0.08 = 1.00$$

- The fourth output has the highest probability, 0.30, so it is the network's most likely answer.

Section 8: Pop Quiz — The Activation on Hidden 1, Neuron 2

- The input activations are now **−2, 1, 3, −4, and 2**, and the weights into Hidden 1, Neuron 2 are **3, −2, 1, 2, and 5**. The activation is their dot product:

$$(-2 \times 3) + (1 \times -2) + (3 \times 1) + (-4 \times 2) + (2 \times 5)$$

$$= -6 + (-2) + 3 + (-8) + 10 = -3$$

- The five products are −6, −2, 3, −8, and 10. The negative products add to −16, the positive products add to 13, and together they give **−3**.
- With linear hidden activations and no bias, the weighted sum is the neuron's final activation.

Section 9: Learning by FITYMI — Backpropagation

- Learning begins with **random weights** and one randomly picked **input-output (I/O) pair**, meaning an input together with its correct output. The input is $x = [-2, 1, 3, -4, 2]$, and

the correct output pdf is:

$$[0,\ 0,\ 0,\ 0,\ 0,\ 1,\ 0]$$

- This target places all of the probability on the sixth of the seven outputs.
- With random weights, the network's output pdf is **wrong**. The network changes its weights to make its output pdf closer to the true pdf, using **SGD**, which for neural networks is now called **backpropagation**.
- This is repeated over and over with **billions of I/O pairs**, and that repeated training is the work done at **data centers**.

Section 10: Python SGD for an Average

- SGD can estimate w, the **average** of this list:

$$[73,\ 90,\ 98,\ 46,\ 400,\ 81,\ 82,\ 78,\ 62,\ 91,\ 61,\ 84,\ 75,\ 84,\ 81,\ 75]$$

- One number, **400**, is a mistake. Including it, the 16 numbers sum to 1561, so the actual average of the list is **about 100**:

$$\frac{1561}{16} \approx 97.56$$

- Without the mistaken 400, the remaining 15 numbers sum to $1561 - 400 = 1161$, so the true average is **about 80**:

$$\frac{1161}{15} = 77.4$$

- The single mistaken data point raises the average by about 20.
- SGD sees the data points one at a time in a **random order**. For ease, the list above is taken to be the random order in which the data arrive, so the estimate w is updated against 73 first, then 90, then 98, and so on through the list.